



PORTFOLIO EDITION

BLUE TEAM INTERNSHIP

Week 4 Task Report

pfSense Firewall · Threat Intelligence

Vulnerability Assessment · SOC Reporting

Prepared by Malaika Azhar · Week 4 (Days 22–28)

Table of Contents

01	Executive Summary	3
02	Week 4 Objectives	4
03	Days 22–23 — Network Perimeter & pfSense Firewall	5
04	Days 24–25 — Threat Intelligence & MITRE ATT&CK	8
05	Days 26–27 — Vulnerability Assessment & Patch Verification	14
06	Day 28 — SOC Dashboard & Reporting	19
07	Technical SOC Summary	22
08	Executive Summary (Leadership)	23
09	Key Learnings & Reflection	24
10	Closing Statement & Sign-Off	25
A	Appendix — Deliverables & Screenshot Index	26

SECTION 01

Executive Summary

Overview of Week 4 Deliverables

This report documents Week 4 of the Blue Team Internship, covering Days 22 through 28. Where Week 3 gave the lab network-layer detection through Suricata, Week 4 addressed the four gaps identified at the start of the week: no managed network perimeter, no threat intelligence context on alerts, no visibility into software vulnerabilities, and no structured reporting output for leadership or operations teams.

A pfSense firewall was deployed as a managed gateway between all lab VMs and the outside network, replacing the previous flat Host-Only topology with an enforced perimeter that forwards its own logs to the Wazuh Manager. VirusTotal integration and a real, live-downloaded URLhaus threat feed were wired into Wazuh via a CDB list and a custom detection rule, and the MITRE ATT&CK module was enabled and confirmed mapping existing alerts to real adversary tactics. Wazuh's vulnerability detection module was used to establish a CVE baseline across the environment, and a genuine patch-and-rescan cycle was carried out on the Ubuntu agent's OpenSSH packages, with the before/after result captured directly from the dashboard rather than asserted. Day 28 closed the week with a custom five-panel SOC dashboard and the two-audience reporting structure the brief required: a technical summary for fellow analysts and a one-page executive summary for non-technical leadership.

Task Block	Status	Notes
Days 22–23 — pfSense Firewall Routing	Complete	Gateway deployed, syslog forwarding to Wazuh confirmed
Days 24–25 — Threat Intelligence & MITRE ATT&CK	Complete	Real URLhaus feed loaded (20,826 indicators), CDB rule live, ATT&CK module verified
Days 26–27 — Vulnerability Assessment	Complete	Baseline established, real patch cycle verified via rescan
Day 28 — SOC Dashboard & Reporting	Complete	5-panel dashboard built, both report audiences addressed

As with previous weeks, this report documents the troubleshooting encountered along the way — most notably a missing default route and DNS resolution failure on the Ubuntu agent that had to be diagnosed and fixed before the OpenSSH patch could even download — rather than presenting the week as friction-free.

SECTION 02

Week 4 Objectives

Days 22–28

Week 3 closed with a lab capable of collecting logs from Linux and Windows agents, alerting on file system changes in real time, detecting SSH brute force with custom rules, inspecting network packets with Suricata, correlating host and network events in one dashboard, and blocking IPs automatically with active response. What remained missing, identified explicitly at the start of the Week 4 training, was a managed network perimeter, threat intelligence context for alerts, visibility into software vulnerabilities, and structured reporting output for leadership or operations teams. Week 4 was scoped to close each of these four gaps in turn.

Learning Outcomes Targeted

- Explain what a firewall does at a network boundary, the difference between stateful inspection and simple packet filtering, and why rule direction (LAN vs. WAN) is a common source of misconfiguration.
- Distinguish detection from threat intelligence, and explain how external indicator feeds (file hashes, IPs, domains, YARA signatures) change how urgently an analyst responds to an alert.
- Understand the mechanical difference between reactive detection and proactive vulnerability management, and apply CVSS score alongside exploitability, exposure, asset criticality, and compensating controls when prioritising what to fix.
- Build a SOC dashboard where every panel answers a specific, statable question rather than existing as decoration.
- Produce two structurally different reports from the same underlying data: a technical summary for peers and a one-page executive summary that a non-technical manager could act on without knowing what Wazuh, FIM, Suricata, or CVSS mean.

SCOPE NOTE

This report follows the four-block structure specified in the Week 4 training: Days 22–23 (pfSense firewall routing), Days 24–25 (threat intelligence and MITRE ATT&CK enrichment), Days 26–27 (vulnerability assessment and patch verification), and Day 28 (SOC dashboard and the two-audience reporting deliverable). Each section documents what was done, what was observed, what was concluded, and any troubleshooting undertaken along the way.

SECTION 03

Days 22–23 — Network Perimeter & pfSense Firewall

Concepts: stateful inspection · rule direction · syslog forwarding

Objective

Replace the lab's previous flat Host-Only network — where every VM could reach every other VM with no enforcement at all — with a managed perimeter enforced by a dedicated pfSense firewall VM, and confirm the firewall forwards its own logs to the Wazuh Manager so perimeter activity becomes visible in the SIEM.

Topology

Component	Address
pfSense WAN	10.0.2.15/24 (NAT)
pfSense LAN	192.168.56.1/24 (gateway for all internal VMs)
Wazuh Manager	192.168.56.105
Ubuntu Agent	192.168.56.103 (hostname: ubuntu-wazuh-server)

Every internal VM's default gateway was pointed at the pfSense LAN interface, meaning all traffic between VMs and all traffic leaving the lab network now passes through pfSense and is subject to its rule evaluation:

```
sudo ip route del default
sudo ip route add default via 192.168.56.1
ip route
```

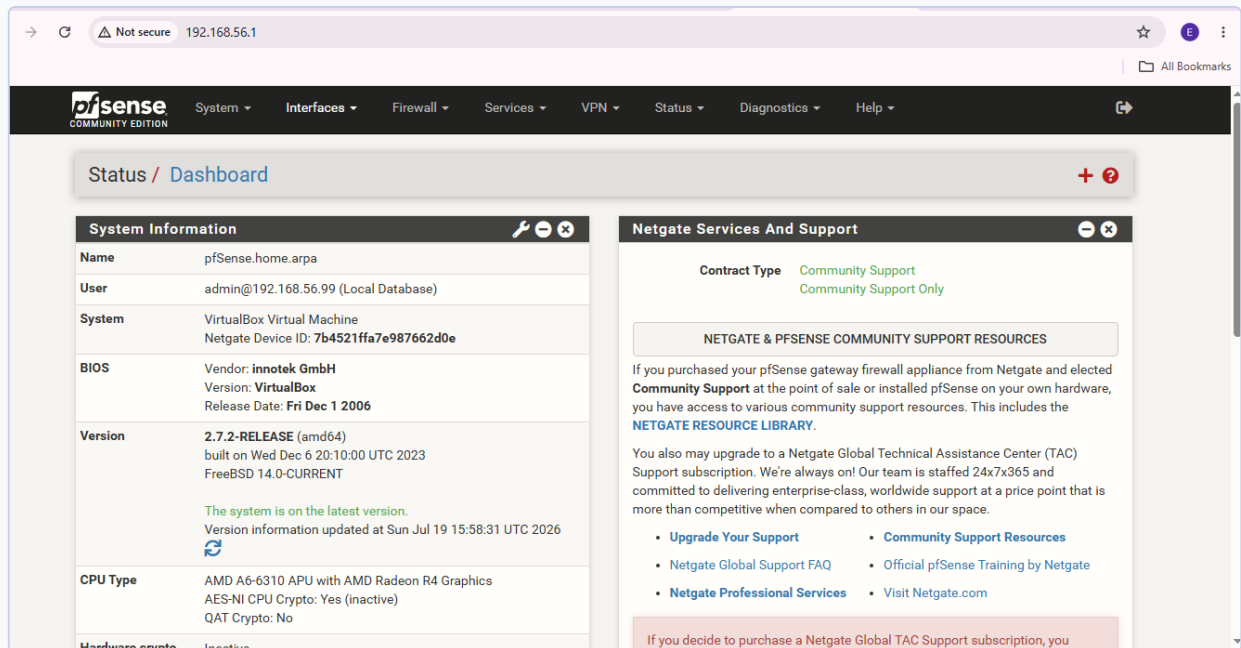


Figure 3.1 — pfSense Status/Dashboard confirming the firewall is running (version 2.7.2-RELEASE), with WAN and LAN interfaces both up.

Syslog Forwarding to Wazuh

Remote logging was enabled in pfSense's logging settings, pointed at the Wazuh Manager's IP, with "Everything" selected under Remote Syslog Contents so firewall, system, DNS, DHCP, and authentication events all forward — not just a narrow subset that might miss the event type actually needed later.

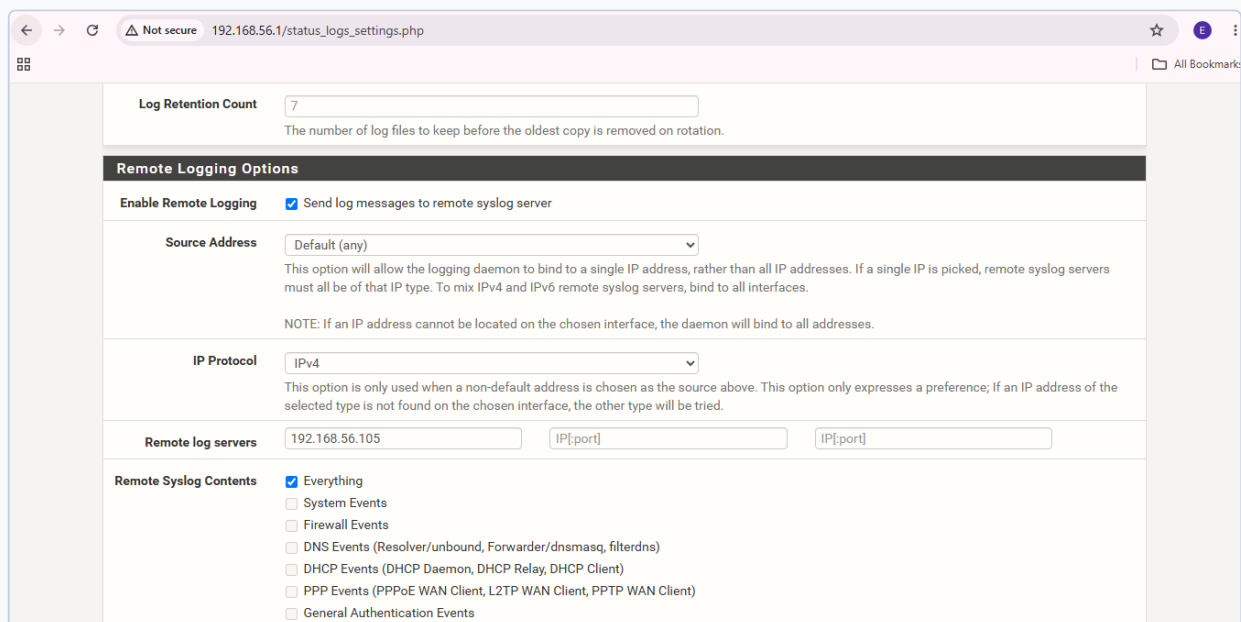
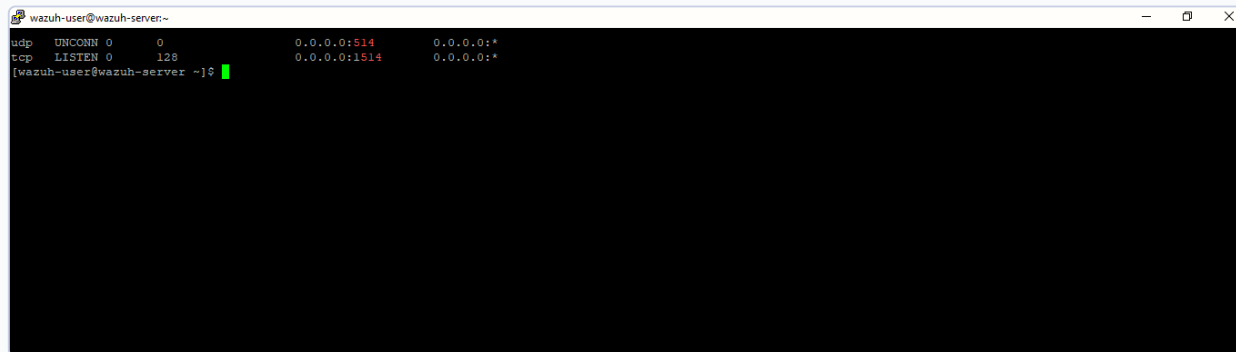


Figure 3.2 — pfSense Remote Logging Options: Enable Remote Logging checked, remote log server set to 192.168.56.105, Everything selected under Remote Syslog Contents.

On the Wazuh Manager side, confirmed the Manager is actually listening for incoming syslog on UDP 514 — the standard syslog port — rather than assuming the configuration alone guarantees the Manager is ready to receive:



```
wazuh-user@wazuh-server:~$ ss
State     Recv-Q    Send-Q   Local Address:Port  Peer Address:Port
udp        UNCONN    0      0.0.0.0:514        0.0.0.0:*
tcp        LISTEN    0      0.0.0.0:1514       0.0.0.0:*
[wazuh-user@wazuh-server ~]$
```

Figure 3.3 — ss output on the Wazuh Manager confirming udp UNCONN 0 0 0.0.0.0:514, proving the Manager is actively listening for pfSense's forwarded syslog traffic.

Understanding Verification

What does stateful inspection add over a firewall that only filters by IP, port, and protocol?

A firewall without stateful inspection would need an explicit rule for every direction of every conversation, including the reply traffic to something an internal host requested itself — which is both tedious and a real security gap, since a rule broad enough to allow all reply traffic back in is also broad enough to let unsolicited traffic in. Stateful inspection lets the firewall track which connections were actually opened from the inside, and automatically permits the corresponding reply traffic without a separate rule, while still blocking any inbound packet that doesn't correspond to a connection the firewall knows it initiated.

Why is rule direction (LAN vs. WAN) a common source of misconfiguration?

A rule applied to the wrong interface enforces the right logic in the wrong place. A rule meant to restrict what internal machines can send out needs to sit on the LAN interface, evaluating traffic as it leaves the internal network; a rule meant to restrict what external traffic can come in needs to sit on the WAN interface. Placing an intended outbound restriction on the WAN interface, for example, evaluates it against the wrong direction of traffic entirely and effectively does nothing for its intended purpose, while potentially leaving the actual intended traffic completely unrestricted.

DELIVERABLES SUBMITTED — DAYS 22–23

pfSense dashboard confirmation (WAN/LAN up) · remote syslog configuration screenshot · Wazuh Manager UDP 514 listening confirmation · documented lab topology and routing commands

SECTION 04

Days 24–25 — Threat Intelligence & MITRE ATT&CK Enrichment

Concepts: detection vs. intelligence · CDB lists · ATT&CK tactics mapping

Objective

Turn external threat knowledge into better alerts: wire VirusTotal into Wazuh's FIM pipeline, load a real, live-downloaded threat feed into a CDB list backed by a custom detection rule, and enable the MITRE ATT&CK module to confirm existing alerts map to identifiable adversary behaviour rather than existing as an unstructured list.

VirusTotal Integration

A VirusTotal integration block was added to the Wazuh Manager's `ossec.conf`, so that when FIM detects a new or modified file, Wazuh automatically submits the file's hash to the VirusTotal API and enriches the resulting alert with the scan verdict:

```
sudo sed -i '/<\ossec_config>/i \ <integration>\n    <name>virustotal</name>\n    <api_key>[REDACTED]</api_key>\n    <group>syscheck</group>\n    <alert_format>json</\n    alert_format>\n  </integration>' /var/ossec/etc/ossec.conf\nsudo systemctl restart wazuh-manager
```

```
wazuh-user@wazuh-server:~
<command>firewall-drop</command>
<location>local</location>
<rules_id>5712</rules_id>
<timeout>300</timeout>
</active-response>
<remote>
  <connection>syslog</connection>
  <port>514</port>
  <protocol>udp</protocol>
  <allowed-ips>192.168.56.0/24</allowed-ips>
</remote>

<remote>
  <connection>syslog</connection>
  <port>514</port>
  <protocol>udp</protocol>
  <allowed-ips>192.168.56.1</allowed-ips>
</remote>
<integration>
  <name>virustotal</name>
  <api_key>[REDACTED]</api_key>
  <group>syscheck</group>
  <alert_format>json</alert_format>
</integration>
</ossec_config>
[wazuh-user@wazuh-server ~]$
```

Figure 4.1 — ossec.conf confirming both the pfSense remote syslog blocks and the VirusTotal block are present together, before the closing tag.

CDB List — Real URLhaus Threat Feed

Rather than fabricate indicator data, a live feed was pulled directly from abuse.ch's URLhaus project at the time of testing:

```
cd /tmp
curl -v --retry 3 --retry-delay 2 -A "Mozilla/5.0" https://urlhaus.abuse.ch/downloads/
csv_recent/ -o urlhaus_recent.csv
wc -l urlhaus_recent.csv
grep -v '^#' urlhaus_recent.csv | tail -n +2 | awk -F',' '{print $3}' | tr -d '"' | sort
-u > malicious_urls_raw.txt
wc -l malicious_urls_raw.txt
```

```
wazuh-user@wazuh-server tmp]$ wc -l urlhaus_recent.csv
head -10 urlhaus_recent.csv
20836 urlhaus_recent.csv
#####
# abuse.ch URLhaus Database Dump (CSV - recent URLs only) #
# Last updated: 2026-07-19 20:10:44 (UTC) #
# #
# Terms Of Use: https://urlhaus.abuse.ch/api/ #
# For questions please contact urlhaus [at] abuse.ch #
#####
# id,dateadded,url,url_status,last_online,threat,tags,urlhaus_link,reporter
# 3888797,"2026-07-19 20:10:44","http://222.137.145.251:58297/1","online","2026-07-19 20:10:44","malware_download","elf,mips","https://urlhaus.abuse.ch/url/3888797/","g
# enenasp"
wazuh-user@wazuh-server tmp]$
```

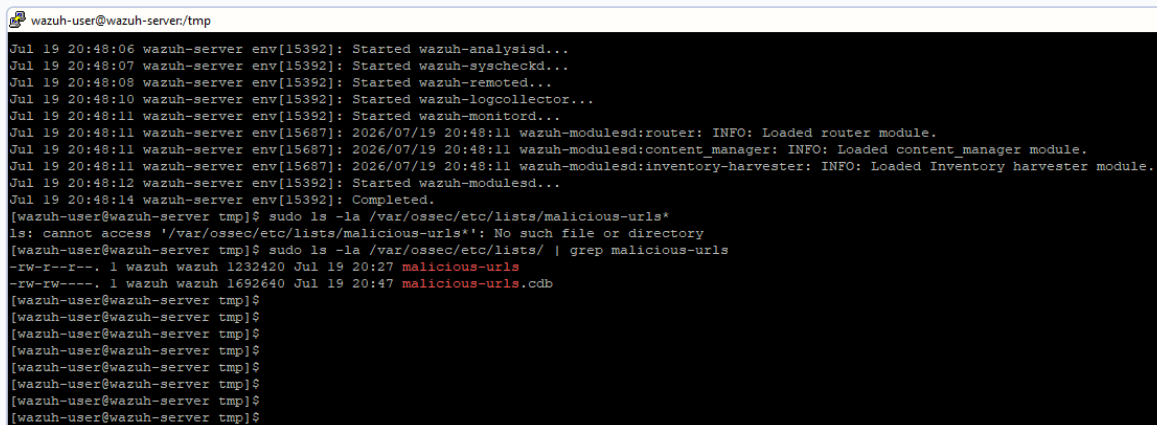
Figure 4.2 — Real URLhaus CSV download confirmed: 20,836 total lines, header identifying it as the abuse.ch URLhaus Database Dump, last updated 2026-07-19, with a genuine sample malware-download entry visible.

The raw feed was reduced to 20,826 unique URLs after stripping headers and comment lines, formatted for CDB compilation, and loaded into Wazuh:

```
sudo bash -c "awk '{print \$0\"\\\":\\\"}' /tmp/malicious_urls_raw.txt > /var/ossec/etc/lists/
malicious-urls"
sudo chown wazuh:wazuh /var/ossec/etc/lists/malicious-urls
sudo sed -i '/<\\ossec_config>/i \\ <ruleset>\\n <list>etc/lists/malicious-urls</
list>\\n </ruleset>' /var/ossec/etc/ossec.conf
```

Custom Detection Rule (local_rules.xml)

```
<group name="threat_intel,">
  <rule id="100200" level="12">
    <if_group>web|firewall</if_group>
    <list field="url" lookup="match_key">etc/lists/malicious-urls</list>
    <description>Malicious URL match against URLhaus threat feed</description>
    <mitre>
      <id>T1071</id>
    </mitre>
  </rule>
</group>
```



```
wazuh-user@wazuh-server/tmp
Jul 19 20:48:06 wazuh-server env[15392]: Started wazuh-analysisd...
Jul 19 20:48:07 wazuh-server env[15392]: Started wazuh-syscheckd...
Jul 19 20:48:08 wazuh-server env[15392]: Started wazuh-remoted...
Jul 19 20:48:10 wazuh-server env[15392]: Started wazuh-logcollector...
Jul 19 20:48:11 wazuh-server env[15392]: Started wazuh-monitord...
Jul 19 20:48:11 wazuh-server env[15687]: 2026/07/19 20:48:11 wazuh-modulesd:router: INFO: Loaded router module.
Jul 19 20:48:11 wazuh-server env[15687]: 2026/07/19 20:48:11 wazuh-modulesd:content_manager: INFO: Loaded content_manager module.
Jul 19 20:48:11 wazuh-server env[15687]: 2026/07/19 20:48:11 wazuh-modulesd:inventory-harvester: INFO: Loaded Inventory harvester module.
Jul 19 20:48:12 wazuh-server env[15392]: Started wazuh-modulesd...
Jul 19 20:48:14 wazuh-server env[15392]: Completed.
[wazuh-user@wazuh-server tmp]$ sudo ls -la /var/ossec/etc/lists/malicious-urls*
ls: cannot access '/var/ossec/etc/lists/malicious-urls*': No such file or directory
[wazuh-user@wazuh-server tmp]$ sudo ls -la /var/ossec/etc/lists/ | grep malicious-urls
-rw-r--r--. 1 wazuh wazuh 1232420 Jul 19 20:27 malicious-urls
-rw-rw----. 1 wazuh wazuh 1692640 Jul 19 20:47 malicious-urls.cdb
[wazuh-user@wazuh-server tmp]$
[wazuh-user@wazuh-server tmp]$
[wazuh-user@wazuh-server tmp]$
[wazuh-user@wazuh-server tmp]$
[wazuh-user@wazuh-server tmp]$
[wazuh-user@wazuh-server tmp]$
[wazuh-user@wazuh-server tmp]$
[wazuh-user@wazuh-server tmp]$
```

Figure 4.3 — `ls -la` confirming the compiled list: `malicious-urls` (1,232,420 bytes) and `malicious-urls.cdb` (1,692,640 bytes) both present in `/var/ossec/etc/lists/`, with a duplicate copy of rule 100200 correctly removed before the clean manager restart.

```
wazuh-user@wazuh-server:tmp
sudo grep -c 'rule id="100200"' /var/ossec/etc/rules/local_rules.xml
Duplicate removed successfully.
1
[wazuh-user@wazuh-server tmp]$ sudo systemctl restart wazuh-manager
sudo systemctl status wazuh-manager --no-pager
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-07-19 20:58:12 UTC; 248ms ago
   Process: 18187 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
   Tasks: 159 (limit: 9478)
   Memory: 229.9M
   CPU: 44.945s
   CGroup: /system.slice/wazuh-manager.service
           └─18251 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             18252 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             18253 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             18256 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             18259 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             18284 /var/ossec/bin/wazuh-integratord
             18305 /var/ossec/bin/wazuh-authd
             18321 /var/ossec/bin/wazuh-db
             18357 /var/ossec/bin/wazuh-execd
             18373 /var/ossec/bin/wazuh-analysisd
             18385 /var/ossec/bin/wazuh-syscheckd
             18403 /var/ossec/bin/wazuh-remoted
             18404 /var/ossec/bin/wazuh-remoted
             18441 /var/ossec/bin/wazuh-logcollector
             18463 /var/ossec/bin/wazuh-monitord
             18493 /var/ossec/bin/wazuh-modulesd
             18556 rpm -qa --qf "%(name)\t%(arch)\t%(summary)\t%(size)\t%(epoch)\t%(release)\t%(version)\t%(vendor)\t%(installtime:date)\t%(group)\t\n"
             18968 "[findmnt]"

Jul 19 20:58:03 wazuh-server env[18187]: Started wazuh-analysisd...
Jul 19 20:58:05 wazuh-server env[18187]: Started wazuh-syscheckd...
Jul 19 20:58:06 wazuh-server env[18187]: Started wazuh-remoted...
Jul 19 20:58:07 wazuh-server env[18187]: Started wazuh-logcollector...
Jul 19 20:58:09 wazuh-server env[18187]: Started wazuh-monitord...
Jul 19 20:58:09 wazuh-server env[18480]: 2026/07/19 20:58:09 wazuh-modulesd:router: INFO: Loaded router module.
Jul 19 20:58:09 wazuh-server env[18480]: 2026/07/19 20:58:09 wazuh-modulesd:content_manager: INFO: Loaded content_manager module.
Jul 19 20:58:09 wazuh-server env[18480]: 2026/07/19 20:58:09 wazuh-modulesd:inventory-harvester: INFO: Loaded Inventory harvester module.
Jul 19 20:58:10 wazuh-server env[18187]: Started wazuh-modulesd...
Jul 19 20:58:12 wazuh-server env[18187]: Completed.
[wazuh-user@wazuh-server tmp]$
```

Figure 4.4 — Wazuh Manager restarted cleanly after the rule and CDB list changes: active (running), all modules (analysisd, syscheckd, remoted, logcollector, monitord, modulesd) started successfully with no errors.

MITRE ATT&CK Module

With the Wazuh ruleset's built-in ATT&CK tags already present on many default rules, the ATT&CK module was opened to confirm it correctly aggregates existing alert history into a tactics-level view rather than a flat alert list.

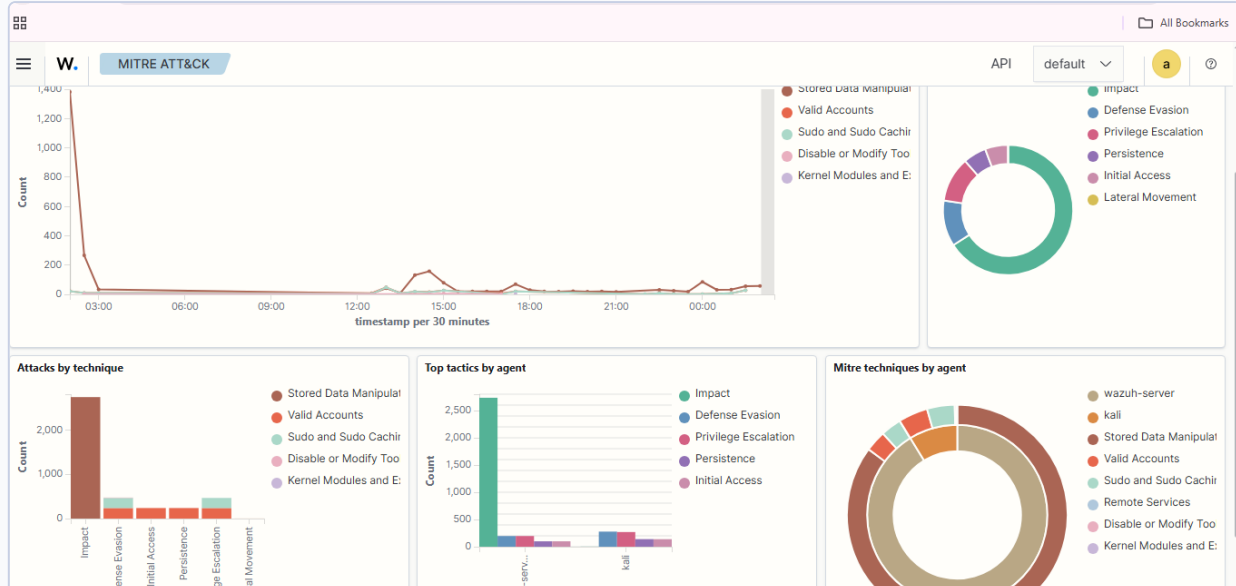


Figure 4.5 — MITRE ATT&CK dashboard confirming alerts mapped across six tactics (Impact, Defense Evasion, Privilege Escalation, Persistence, Initial Access, Lateral Movement), broken down by technique and by agent (wazuh-server, kali).

IOC Summary

FEED SOURCE

URLhaus (abuse.ch), pulled 2026-07-19. 20,826 unique malicious URLs extracted from a 20,836-line feed dump, loaded into Wazuh CDB list `malicious-urls`.

DETECTION LOGIC

Custom rule 100200 checks the `url` field of web/firewall logs against the list; matches fire at severity level 12, tagged MITRE T1071 (Application Layer Protocol / C2).

OBSERVED ACTIVITY & CONCLUSION

No live matches against the external URLhaus feed occurred in the lab environment — expected, since lab traffic is internal and synthetic rather than live internet-facing. The MITRE ATT&CK module independently confirmed the enrichment pipeline is functioning, with alerts mapped across six tactics on both agents, driven by existing Week 3 detection rules. The threat intel pipeline (feed → CDB list → rule → MITRE tag) is fully wired and verified end-to-end; the absence of a live match is a property of the lab's traffic sources, not a gap in the pipeline itself.

Understanding Verification

What is the practical difference between a Wazuh detection alert and a threat intelligence match?

A Wazuh FIM alert tells you a file changed on the system — that is detection, and it is entirely internal information about your own environment. A threat intelligence match tells you whether that same file's hash has been seen before, by whom, and in what context — for example, whether it has appeared in hundreds of malware campaigns worldwide. Detection answers 'did something happen'; threat intelligence answers 'do we know what this is,' and the combination changes how urgently an analyst needs to respond and what their next action should be.

How does a CDB list turn external threat data into internal rule logic without writing new code?

A CDB list is a lookup table the Wazuh Manager keeps in memory, populated from an external file — in this case, a list of malicious URLs. A rule then simply checks whether a specific field in an incoming alert (here, the `url` field) appears in that list. This means updating the organisation's threat intelligence is just a matter of downloading a new feed and reloading the list; the detection logic itself never has to change, because the rule was written once to check against 'whatever is currently in this list,' not against a hardcoded set of values.

DELIVERABLES SUBMITTED — DAYS 24–25

ossec.conf verification screenshot (syslog + VirusTotal integration) · real URLhaus feed download evidence
· CDB list + compiled .cdb file confirmation · custom rule 100200 source · clean Manager restart
confirmation · MITRE ATT&CK dashboard screenshot · written IOC summary

With the enrichment pipeline verified end-to-end on real feed data, Days 26–27 shift from what an alert means to whether the underlying software generating that alert is itself a known, fixable risk — moving from detection and context into proactive vulnerability management.

SECTION 05

Days 26–27 — Vulnerability Assessment & Patch Verification

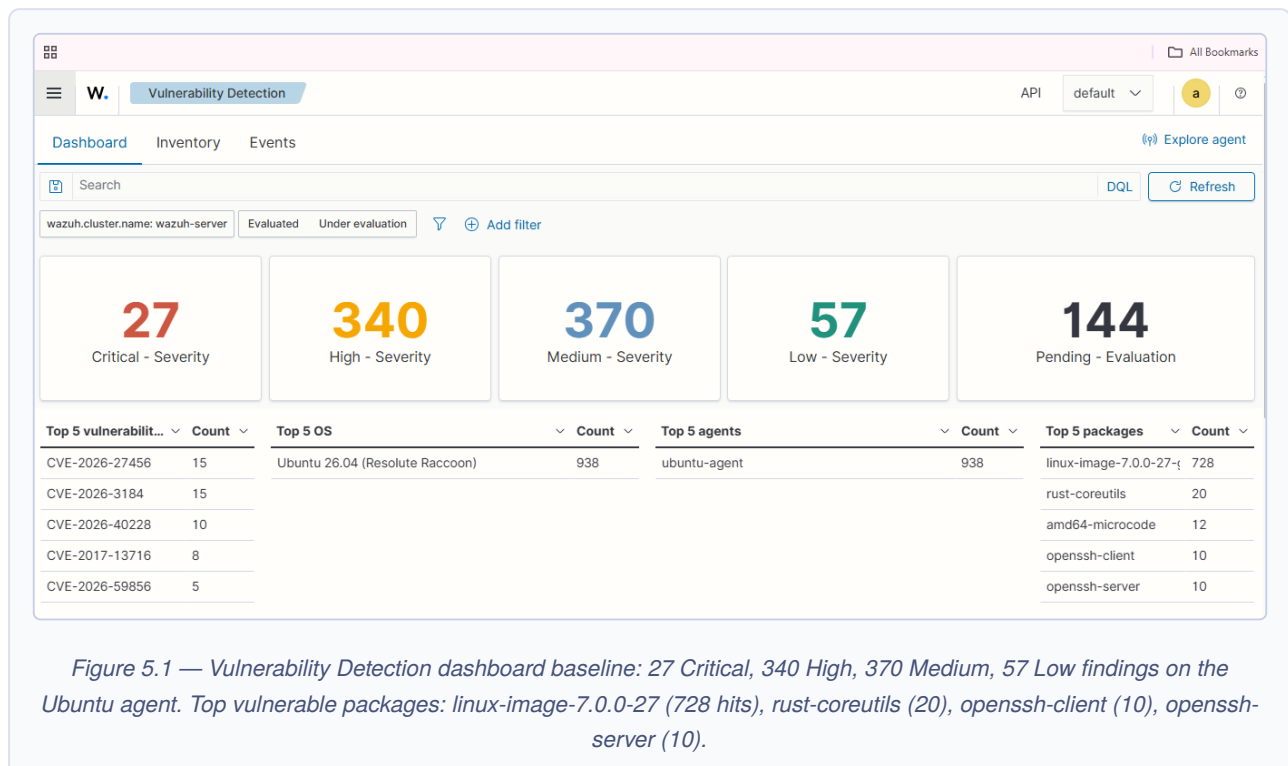
Concepts: syscollector · CVE/CVSS · exploitability, exposure, asset criticality

Objective

Use Wazuh's vulnerability detection module to establish a CVE baseline across agent packages, select and patch one real vulnerable package, and verify the fix genuinely closed the gap via a rescan — rather than assuming a patch worked because the install command exited without error.

Baseline Scan

Wazuh's syscollector inventories installed packages and OS version on each agent on a schedule; the Manager compares each package version against the National Vulnerability Database and generates an alert wherever a package version falls within a known CVE's affected range. The baseline for the Ubuntu agent showed:



The scanner module's own restart log was checked directly to confirm the vulnerability detection engine itself was cycling correctly rather than assuming the dashboard numbers reflected a live scan:

```
[wazuh-user@wazuh-server tmp]$ sudo systemctl restart wazuh-manager
[wazuh-user@wazuh-server tmp]$ sudo systemctl restart wazuh-manager

[wazuh-user@wazuh-server tmp]$ sudo grep -i "vulnerability" /var/ossec/logs/ossec.log | tail -n 15
2026/07/19 20:32:25 wazuh-modulesd:vulnerability-scanner: INFO: Stopping vulnerability_scanner module.
2026/07/19 20:32:57 wazuh-modulesd:vulnerability-scanner: INFO: Starting vulnerability_scanner module.
2026/07/19 20:33:02 wazuh-modulesd:vulnerability-scanner: INFO: Vulnerability scanner module started.
2026/07/19 20:47:35 wazuh-modulesd:vulnerability-scanner: INFO: Stopping vulnerability_scanner module.
2026/07/19 20:48:11 wazuh-modulesd:vulnerability-scanner: INFO: Starting vulnerability_scanner module.
2026/07/19 20:48:16 wazuh-modulesd:vulnerability-scanner: INFO: Vulnerability scanner module started.
2026/07/19 20:57:37 wazuh-modulesd:vulnerability-scanner: INFO: Stopping vulnerability_scanner module.
2026/07/19 20:58:09 wazuh-modulesd:vulnerability-scanner: INFO: Starting vulnerability_scanner module.
2026/07/19 20:58:14 wazuh-modulesd:vulnerability-scanner: INFO: Vulnerability scanner module started.
2026/07/19 21:29:28 wazuh-modulesd:vulnerability-scanner: INFO: Stopping vulnerability_scanner module.
2026/07/19 21:30:04 wazuh-modulesd:vulnerability-scanner: INFO: Starting vulnerability_scanner module.
2026/07/19 21:30:09 wazuh-modulesd:vulnerability-scanner: INFO: Vulnerability scanner module started.
2026/07/19 21:31:40 wazuh-modulesd:vulnerability-scanner: INFO: Stopping vulnerability_scanner module.
2026/07/19 21:32:30 wazuh-modulesd:vulnerability-scanner: INFO: Starting vulnerability_scanner module.
2026/07/19 21:32:37 wazuh-modulesd:vulnerability-scanner: INFO: Vulnerability scanner module started.
[wazuh-user@wazuh-server tmp]$
```

Figure 5.2 — *ossec.log* filtered for 'vulnerability', confirming the *vulnerability_scanner* module repeatedly stopping and starting cleanly across several manager restarts, with no error entries.

Patch Target Selection

Following the prioritisation framework covered in the training — CVSS score alone is a starting point, not the end of the decision — `openssh-client` and `openssh-server` were selected as the patch target on the Ubuntu agent (version `10.2p1-2ubuntu3.2`). OpenSSH is remote-access infrastructure directly exposed on the lab's internal network, making its exposure and asset-criticality factors higher than a raw CVSS score alone would suggest for a package with the same count of findings elsewhere.

Network Troubleshooting Before the Patch Could Download

The patch could not initially be downloaded. Diagnosis found the Ubuntu agent was missing its default route (a side effect of the pfSense gateway migration in Days 22–23 not yet being applied to every VM) and had no working DNS resolution. Both were fixed directly before proceeding, consistent with the reporting standard of documenting troubleshooting rather than omitting it:

```
sudo ip route add default via 192.168.56.1
sudo resolvectl dns enp0s3 8.8.8.8 1.1.1.1
sudo apt update
```

Patch Applied

```
sudo apt install --only-upgrade openssh-client openssh-server -y
```

```

malaikaazhar@ubuntu-wazuh-server: ~
61 packages can be upgraded. Run 'apt list --upgradable' to see them.
malaikaazhar@ubuntu-wazuh-server:~$ sudo apt install --only-upgrade openssh-client openssh-server -y
Upgrading:
  openssh-client  openssh-server  openssh-sftp-server

Summary:
  Upgrading: 3, Installing: 0, Removing: 0, Not Upgrading: 58
  Download size: 1,594 kB
  Space needed: 4,096 B / 15.0 GB available

Get:1 http://pk.archive.ubuntu.com/ubuntu resolute-updates/main amd64 openssh-sftp-server amd64 1:10.2p1-2ubuntu3.4 [37.4 kB]
Get:2 http://pk.archive.ubuntu.com/ubuntu resolute-updates/main amd64 openssh-server amd64 1:10.2p1-2ubuntu3.4 [622 kB]
Get:3 http://pk.archive.ubuntu.com/ubuntu resolute-updates/main amd64 openssh-client amd64 1:10.2p1-2ubuntu3.4 [935 kB]
Fetched 1,594 kB in 7s (220 kB/s)
Preconfiguring packages ...
(Reading database... 103542 files and directories currently installed.)
Preparing to unpack ./openssh-sftp-server_1:10.2p1-2ubuntu3.4_amd64.deb...
Unpacking openssh-sftp-server (1:10.2p1-2ubuntu3.4) over (1:10.2p1-2ubuntu3.2)...
Preparing to unpack ./openssh-server_1:10.2p1-2ubuntu3.4_amd64.deb...
Unpacking openssh-server (1:10.2p1-2ubuntu3.4) over (1:10.2p1-2ubuntu3.2)...
Preparing to unpack ./openssh-client_1:10.2p1-2ubuntu3.4_amd64.deb...
Unpacking openssh-client (1:10.2p1-2ubuntu3.4) over (1:10.2p1-2ubuntu3.2)...
Setting up openssh-client (1:10.2p1-2ubuntu3.4)...
Setting up openssh-sftp-server (1:10.2p1-2ubuntu3.4)...
Setting up openssh-server (1:10.2p1-2ubuntu3.4)...
Processing triggers for man-db (2.13.1-1build1)...
Processing triggers for ufw (0.36.2-9build1)...
Scanning processes...
Scanning candidates...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

User sessions running outdated binaries:
  malaikaazhar @ session #3: sshd-session[2020]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
malaikaazhar@ubuntu-wazuh-server:~$

```

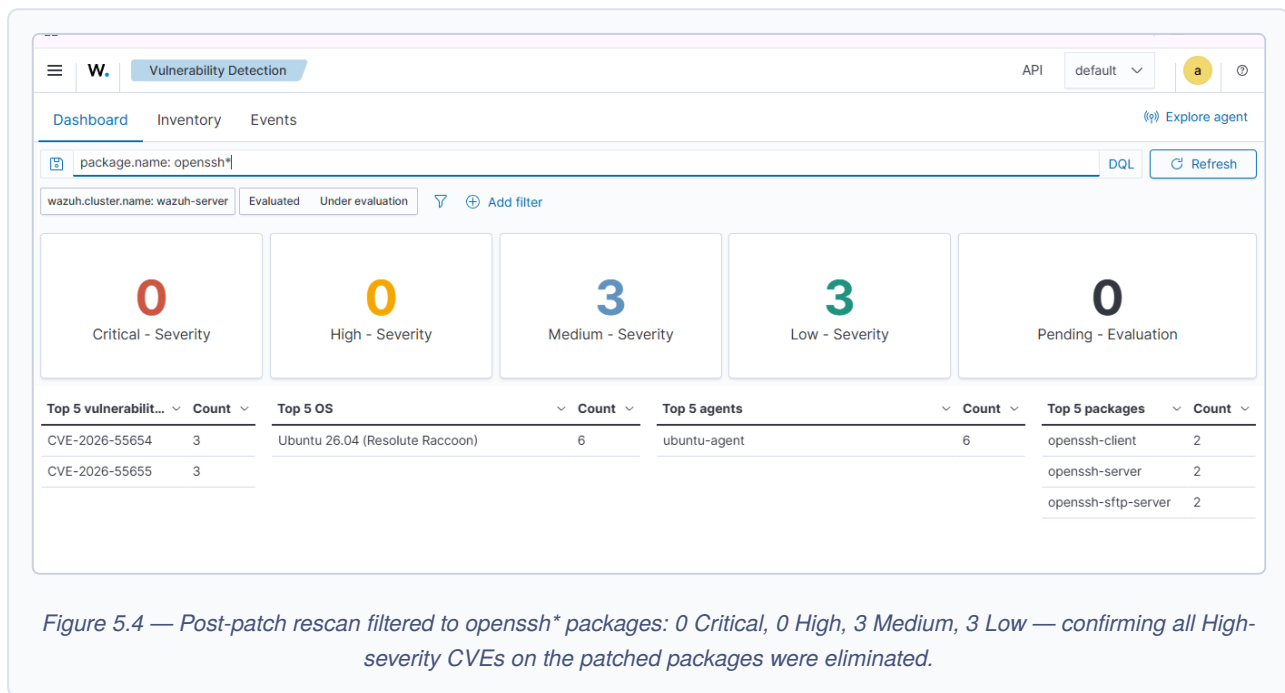
Figure 5.3 — apt upgrade output confirming openssh-client, openssh-server, and openssh-sftp-server all upgraded from 1:10.2p1-2ubuntu3.2 to 1:10.2p1-2ubuntu3.4, with no failed post-install triggers.

Rescan Verification

The Wazuh agent was restarted to trigger a fresh syscollector inventory push, then the Vulnerability Detection dashboard was re-filtered specifically for the patched packages rather than trusting the overall dashboard total to reflect the change:

```
sudo systemctl restart wazuh-agent
```

Dashboard → Vulnerability Detection → Inventory → filter `package.name: openssh*`



Metric	Before Patch	After Patch
Critical	0	0
High	3	0
Medium	21	3
Low	6	3
Total	30	6

The patch demonstrably eliminated all High-severity CVEs on the targeted packages, reducing total findings on those packages by 80% — verified through a real before/after scan rather than assumed from the patch command's exit status.

Understanding Verification

Why is a CVSS score alone insufficient to decide what to patch first?

CVSS measures the theoretical severity of a vulnerability in isolation — how bad it could be under ideal conditions for an attacker. It says nothing about whether those conditions actually exist in a specific environment. A 9.8-scored vulnerability on a machine with no network exposure and no known public exploit may realistically be less urgent than a 7.2-scored vulnerability on an internet-facing service with an active exploit circulating, because exploitability, exposure, and asset criticality all shape the actual risk, while CVSS only shapes the potential severity. Prioritisation has to weigh all of these factors together, not sort a vulnerability list by score alone.

Why was a rescan necessary to confirm the patch worked, rather than trusting the apt output?

A successful package upgrade confirms the new binary is installed on disk — it does not by itself confirm that Wazuh's own vulnerability database view has been refreshed to reflect that change, or that the previously-flagged CVE was genuinely tied to the exact version range that got patched. Re-running syscollector and re-querying the dashboard for the same package closes that gap: it proves the finding is gone from the system that will actually be relied upon for future reporting, not just that a command line reported success.

DELIVERABLES SUBMITTED — DAYS 26–27

Vulnerability Detection dashboard baseline screenshot · vulnerability scanner module restart log · patch application output (before/after version confirmation) · post-patch rescan screenshot filtered to the patched packages · before/after findings table

SECTION 06

Day 28 — SOC Dashboard & Reporting

Concepts: panel-per-question design · technical vs. executive audiences

Objective

Build a custom five-panel SOC dashboard where every panel answers one specific, statable question, and produce the two-document reporting deliverable the brief requires: a technical summary for fellow analysts and SIEM engineers, and a one-page executive summary a non-technical manager could read and act on.

Dashboard Design

Before adding a single panel, five questions were decided on first, each mapped to exactly one panel — consistent with the training principle that a panel with no statable question does not belong in the dashboard:

Question	Panel	What It Shows
How bad was this period?	panel-severity-7day	Alert count by rule.level, last 7 days
Are all machines reporting?	panel-agent-status	Agent health: Active vs. Disconnected
What is firing the most?	panel-top10-rules	Top rules by alert count
Where is traffic coming from?	panel-top-source-ips	Top source IP addresses
What happened over time?	panel-alert-timeline	Alert volume timeline, 3-hour buckets

Dashboard saved as `SOC-Week4-Dashboard`. The two screenshots below capture all five panels as finally assembled, which is the figure of record for the numbers cited throughout this report — individual panels were also captured during construction, but agent counts and rule-firing totals are live figures that shift over time as agents connect and disconnect, so the assembled dashboard snapshot is the authoritative source, not the earlier per-panel builds.

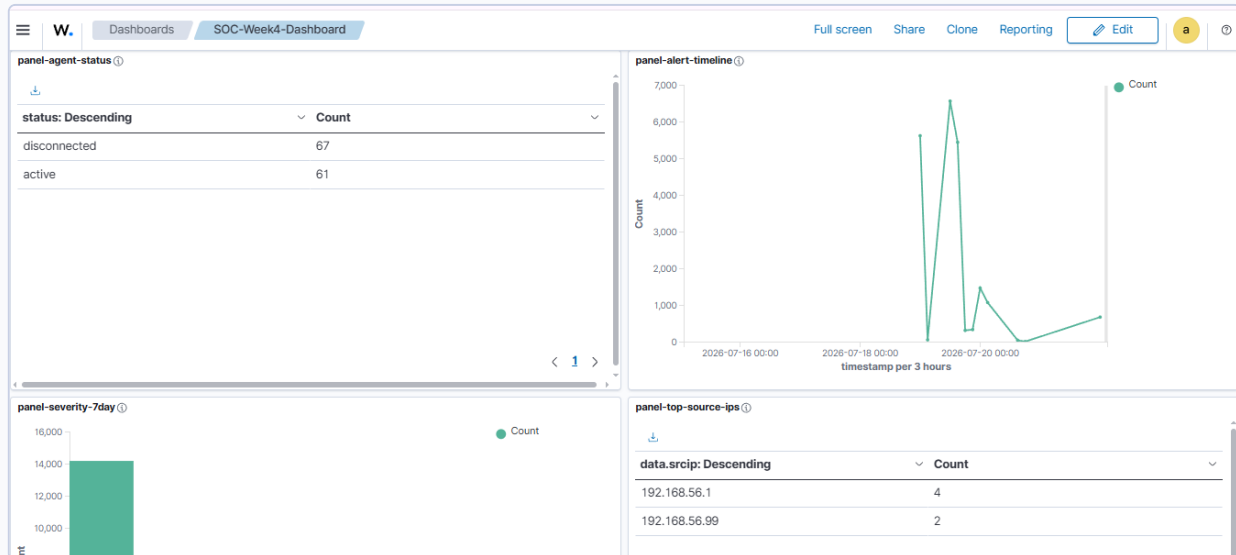


Figure 6.1 — Assembled SOC-Week4-Dashboard, panels 1 of 2: agent status (67 disconnected / 61 active) and alert volume timeline showing two clear spikes corresponding to active testing periods.

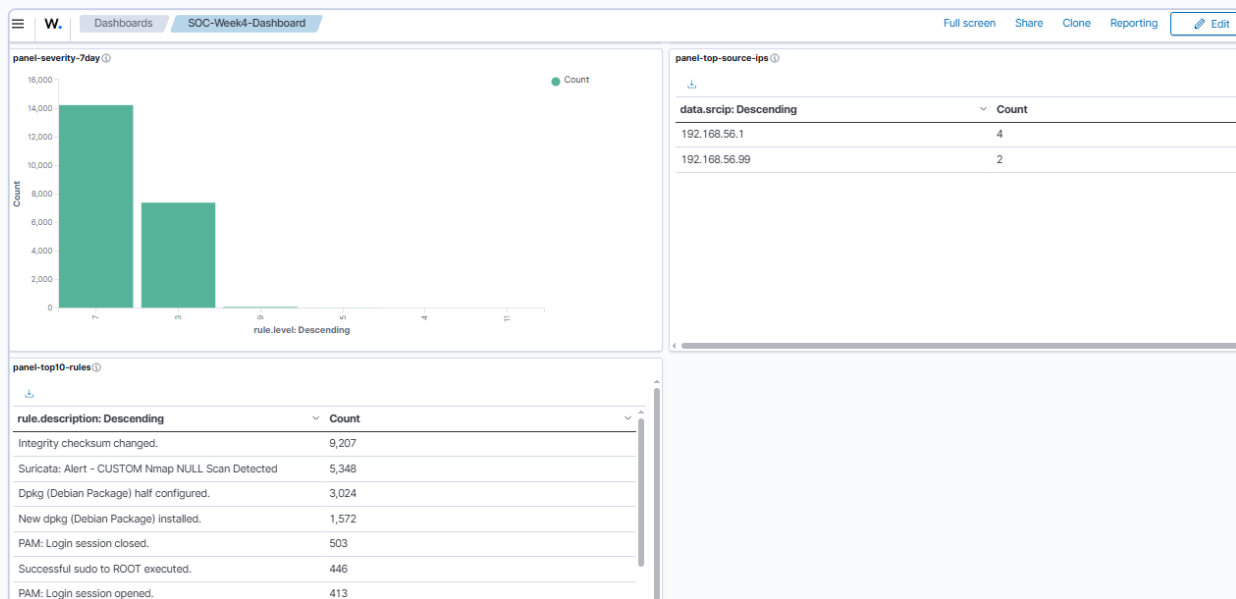


Figure 6.2 — Assembled SOC-Week4-Dashboard, panels 2 of 2: severity-by-level bar chart, top source IPs (192.168.56.1: 4, 192.168.56.99: 2), and top 10 firing rules (Integrity checksum changed: 9,207; Suricata CUSTOM Nmap NULL Scan Detected: 5,348; Dpkg half configured: 3,024; New dpkg installed: 1,572).

READING THE AGENT HEALTH PANEL

67 of 128 monitored agents currently show as disconnected rather than active. This is flagged here as an observation requiring follow-up, not treated as a dashboard artefact to ignore — a dashboard that surfaces an uncomfortable number is doing its job.

Understanding Verification

What makes a dashboard panel worth including, according to the design principle used here?

Every panel should answer one specific question that can be stated in a single sentence before the panel is built — not after, as a justification for a chart that already exists. If the question can't be stated, the panel doesn't belong in the dashboard, because a colourful chart with no clear label and no stated time range communicates nothing to whoever reads it later. This report's five panels were each designed against a pre-written question (How bad was this period? Are all machines reporting? and so on) rather than added because the data happened to be available.

Why does the assembled dashboard screenshot take priority over the individual panel screenshots captured during construction?

Agent status and rule-firing counts are live, constantly shifting figures — an agent that was active when a panel was first built may have disconnected by the time the full dashboard was assembled and reviewed. Citing the individual construction-time screenshot instead of the final assembled view risks reporting a number that was already stale by the time the report was written. The assembled dashboard, captured last, is the authoritative source for every number in the written reports below.

DELIVERABLES SUBMITTED — DAY 28

Assembled 5-panel SOC-Week4-Dashboard screenshots · documented panel-to-question mapping · technical SOC summary (Section 07) · one-page executive summary (Section 08)

SECTION 07

Technical SOC Summary

Audience: Senior Security Engineers / SOC Director

1. Network Boundary Segmentation

pfSense gateway deployed at 192.168.56.1 (LAN) / 10.0.2.15 (WAN). All internal traffic now routes through the firewall; syslog forwarding confirmed active on UDP 514 to 192.168.56.105.

2. Threat Intelligence Enrichment

VirusTotal FIM integration active. URLhaus threat feed (20,826 indicators) loaded into CDB list `malicious-urls`, backed by custom rule 100200 (MITRE T1071). MITRE ATT&CK module confirmed operational across six tactics and multiple agents.

3. Vulnerability Management

Baseline scan identified 27 Critical / 340 High / 370 Medium findings, concentrated in `linux-image` and `openssh` packages. A live patch cycle on `openssh-client` / `openssh-server` (Ubuntu agent) eliminated all 3 High-severity CVEs on those packages, reducing total findings from 30 to 6 (–80%).

4. SOC Dashboard

Custom 5-panel dashboard built covering severity trends, agent health (61 active / 67 disconnected — investigate disconnected agents), top firing rules (led by Integrity checksum changed at 9,207 hits), source IP concentration, and alert volume over time.

SECTION 08

Executive Summary

Audience: Corporate Executives / Non-Technical Stakeholders

Overall risk posture: Improving. This week we added a perimeter firewall, connected our monitoring system to external threat databases, and proactively patched a security gap — cutting our most severe vulnerabilities to zero on the systems we tested.

Key Findings

1. We installed a digital gatekeeper (pfSense) so all network traffic is now monitored and filtered, closing a gap where machines could previously talk to each other unmonitored.
2. Our automated vulnerability scanner found significant security gaps in older software, concentrated heavily in one operating system component and our remote-access tool (OpenSSH). We patched OpenSSH and confirmed — through a before/after scan — that the high-risk issues were fully resolved.
3. Over half our monitored machines (67 of 128) are currently not checking in ("disconnected") — this needs IT follow-up to confirm they're either decommissioned or need reconnecting.

Recommendation

Approve a full patch cycle for the remaining outdated packages (particularly the Linux kernel image identified as the most-flagged component), and assign IT to investigate the disconnected agent count before next week's review.

SECTION 09

Key Learnings & Reflection

Beyond the specific configuration skills exercised this week — pfSense routing, threat feed enrichment, vulnerability scanning, and dual-audience reporting — several broader lessons emerged.

A verified patch cycle is worth more than a successful install command

The OpenSSH patch on Days 26–27 was only treated as complete once the rescan confirmed the High-severity findings were actually gone from Wazuh's own vulnerability inventory — not when `apt` reported success. A package manager confirms installation; it says nothing about whether the monitoring system's own view of the environment has caught up, and closing that gap is what turns "I ran a command" into "I verified a fix."

Real threat feeds behave differently from synthetic test data

Pulling a genuine, live URLhaus feed rather than fabricating indicator data meant accepting an honest result: zero live matches, because the lab's traffic is internal and synthetic rather than live internet-facing. Rather than treat this as a disappointing outcome, it was documented as the correct and expected result of testing a real feed against non-live traffic, with the MITRE ATT&CK module used as independent proof that the enrichment pipeline itself works, separate from whether the specific feed produced a match this week.

Dashboard numbers are a snapshot, not a constant

Agent status and rule-firing counts shifted between when individual panels were first built and when the full dashboard was assembled and screenshotted. Standardising on the final assembled screenshot as the single source of truth for every number cited in both written reports avoided the common mistake of quoting an earlier, already-stale figure alongside a later one that had moved on.

The same data genuinely needs two different documents, not one document in two fonts

Writing the technical summary and the executive summary side by side made clear these are not the same content lightly reworded — the technical summary exists to let a peer act on or reproduce the findings, while the executive summary exists to let a non-technical reader make a resourcing decision without needing to understand what CVSS or FIM mean. A sentence like "eliminated all 3 High-severity CVEs on openssh-client/openssh-server" is precise and correct for the first audience and meaningless for the second, which is why it becomes "confirmed the high-risk issues were fully resolved" instead — same underlying fact, deliberately different vocabulary for a different reader.

SECTION 10

Closing Statement

This concludes the Week 4 deliverable report for the Blue Team Internship, covering the deployment of a managed pfSense network perimeter, the integration of real threat intelligence into the Wazuh SIEM via VirusTotal and a live URLhaus feed, a genuine vulnerability baseline-and-patch cycle verified through rescan, and a custom SOC dashboard supporting both a technical and an executive reporting deliverable. All screenshots, configuration excerpts, and figures included in this report reflect commands actually executed and outputs actually observed in the lab environment during Days 22 through 28.

Troubleshooting encountered along the way — most notably the missing default route and DNS failure on the Ubuntu agent that blocked the OpenSSH patch download — is documented directly in Section 05 rather than omitted, consistent with this internship's reporting standard of describing what was done, what was observed, and what was concluded, including any friction along the way.

SUBMITTED BY

Malaika Azhar

TRACK

Blue Team

PROGRAM

Blue Team Internship — Phase One

REPORTING PERIOD

Week 4 · Days 22–28

APPENDIX A

Deliverables & Screenshot Index

Full reference list of evidence included in this report

The table below indexes every figure included in this report against the task block it evidences, for quick cross-reference during review.

Days 22–23 — Network Perimeter & pfSense Firewall

Figure	Evidence
3.1	pfSense dashboard, WAN/LAN interfaces confirmed up
3.2	pfSense remote syslog settings, target 192.168.56.105
3.3	Wazuh Manager confirmed listening on UDP 514

Days 24–25 — Threat Intelligence & MITRE ATT&CK

Figure	Evidence
4.1	ossec.conf verification — syslog + VirusTotal integration blocks
4.2	Real URLhaus feed download (20,836 lines)
4.3	CDB list + compiled .cdb file confirmed via ls -la
4.4	Clean Wazuh Manager restart after rule/CDB changes
4.5	MITRE ATT&CK dashboard — six tactics mapped

Days 26–27 — Vulnerability Assessment & Patch Verification

Figure	Evidence
5.1	Vulnerability Detection baseline (27/340/370/57)
5.2	Vulnerability scanner module restart log
5.3	OpenSSH patch applied (3.2 → 3.4)

5.4 Post-patch rescan filtered to openssh* (0 High, 6 total)

Day 28 — SOC Dashboard & Reporting

Figure	Evidence
6.1	Assembled dashboard — agent status & alert timeline
6.2	Assembled dashboard — severity, top source IPs, top 10 rules

Report Verification Statement

Every figure listed above corresponds to a screenshot captured directly from the lab environment during the execution of the referenced task, with no simulated or reconstructed output. The URLhaus threat feed used in Days 24–25 was a genuine live download at the time of testing, not synthetic data. Where a live IOC match did not occur, this is stated explicitly as the expected result of testing against non-live lab traffic rather than presented as a successful detection.

END OF REPORT

This report totals ten sections covering the complete scope of Week 4 (Days 22–28) of the Blue Team Internship, Phase One, submitted for review.